PART I · CHAPTER 2

Transformer Architecture: A Performance Engineer's Deep Dive

Attention, normalization, and feed-forward networks seen as memory and compute problems — with full FLOP and KV cache derivations for LLaMA-3 70B from first principles.

Book: AI/ML Infrastructure from Silicon to Scale — V2 Edition | **Author:** Venkat Vinjam | **Cross-ref:** Ch 4: FlashAttention Ch 6: Inference serving Ch 11: KV cache

> *"The transformer is not an algorithm. It is a data movement problem wearing an algorithm's clothes. Every optimization in LLM infrastructure is an optimization for how we move Q, K, V, and weight matrices through the memory hierarchy."*
>
> — Performance Engineering Perspective

A performance engineer who cannot derive the FLOP and memory budget of a transformer forward pass from the model config cannot make informed decisions about hardware, batching, parallelism, or quantization. This chapter gives you that derivation — every formula from first principles, verified with LLaMA-3 70B numbers from the published model card.

2.1 Why Performance Engineers Must Understand Transformers

Every GPU workload in modern AI is a transformer workload. The model config — layers, hidden dim, heads, FFN multiplier, vocab size, sequence length, attention variant — directly and quantifiably determines hardware requirements. Without deriving these from first principles you cannot right-size a cluster, choose a quantization format, set tensor parallelism degree, or engineer an SLA.

What you must be able to derive from a model card:

(1) Total parameter count and memory in any precision. (2) FLOPs per token for prefill vs decode. (3) KV cache bytes per token per request. (4) Arithmetic intensity of each operation class. (5) Which operations are compute- vs memory-bound at each batch size — before running a single experiment.

2.2 Attention as Matrix Multiplication

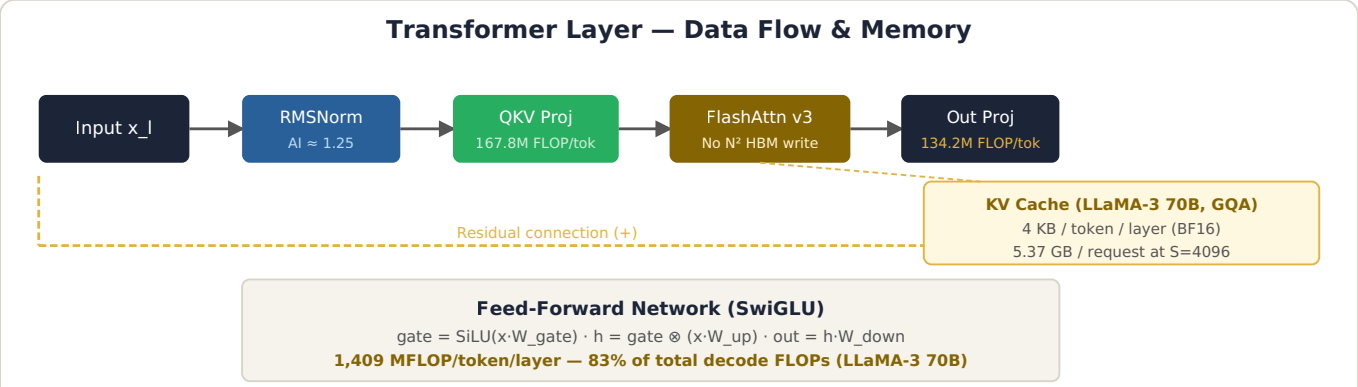
Scaled dot-product attention: **Attention(Q, K, V) = softmax($QK^T/\sqrt{d_k}$) × V**. For a performance engineer, this is two matrix multiplies separated by a softmax with specific memory access patterns.

```
ATTENTION FLOP AND MEMORY ANALYSIS — single head, single layer:
  S = sequence length, d = head dimension = d_model / H_q

  QK^T:     2 × S × d × S = 2S²d FLOPs     (S×d input, S×S output)
  Softmax:  ~5 × S²  FLOPs                  (≈ 1% of total — negligible)
  AV:       2 × S × S × d = 2S²d FLOPs     (S×S attention, S×d output)
  Total per head: 4S²d FLOPs

  HBM traffic (standard attention, BF16):
    Writes: S×S attention matrix twice (forward + backward)
    At S=4096, d=128: S×S matrix = 4096² × 2 bytes = 33.6 MB per head
    For H_q=64 heads: 64 × 33.6 MB = 2.15 GB just for attention scores!

    This is why standard attention is always memory-bound:
    AI = 4S²d / (4Sd + 3S²) / 2 ≈ d/3 for large S
    At d=128: AI ≈ 43 FLOP/byte — deeply memory-bound on H100 (ridge=295)
    CRITICAL: AI does NOT grow with S — standard attention is always bandwidth-bound.
    FlashAttention eliminates the S×S HBM writes entirely (→ Ch 4 §4.8).
```



2.1 — One transformer decoder block. At decode batch=1, all operations are memory-bandwidth-bound. At large-batch prefill, QKV and FFN GEMMs become compute-b

Figure 2.1 — Transformer layer anatomy with FLOPs per token and memory access patterns. The FFN dominates at 83% of FLOPs during decode (S=1). The KV cache grows by 4 KB per token per layer for LLaMA-3 70B with GQA (H_kv=8), accumulating to 5.37 GB per request at 4096-token context.

2.3 Multi-Head Attention: Memory and Compute

```
QKV PROJECTION FLOPS (LLaMA-3 70B):
  Config: d_model=8192, H_q=64, H_kv=8 (GQA), d_head=128

  Q proj: 2 × 8192 × (64×128)  = 2 × 8192 × 8192 = 134.2M FLOPs/token
  K proj: 2 × 8192 × (8×128)   = 2 × 8192 × 1024 =  16.8M FLOPs/token
  V proj: same as K             =                    16.8M FLOPs/token
  O proj: 2 × 8192 × 8192      =                   134.2M FLOPs/token
  Total:  302.0M FLOPs per token per layer

KV CACHE MEMORY (the per-request serving cost):
  Per token per layer (BF16):
    2 (K+V) × H_kv × d_head × 2 bytes = 2 × 8 × 128 × 2 = 4,096 bytes = 4 KB

  Full model (L=80 layers, S=4096 ctx, BF16):
    KV = 2 × 80 × 8 × 128 × 4096 × 2 = 5.37 GB per concurrent request

  Impact: on H100 (80 GB), with 35 GB weights (TP=2) + 10 GB overhead:
    Available for KV: 35 GB → 35 / 5.37 = 6 max concurrent requests (BF16)
    INT8 KV: 2.69 GB/request → 13 concurrent requests (2× capacity)
```

2.4 RMSNorm vs. LayerNorm

```
LAYERNORM:  y = ((x - μ) / σ) × γ + β
  Requires mean and variance: 2 passes over data
  FLOPs/element: ~8   Arithmetic Intensity: ~1.5 FLOP/byte → memory-bound

RMSNORM (LLaMA-3, Mistral, Gemma — modern standard):
  y = x / RMS(x) × γ,  where RMS(x) = √(mean(x²))
  No mean subtraction — one fewer statistics pass
  FLOPs/element: ~5   Arithmetic Intensity: ~1.25 FLOP/byte → memory-bound

WHY RMSNorm won in production:
  1. 40% fewer FLOPs — measurable at 80-layer models, 10K tokens/s serving
  2. Simpler gradient: no mean subtraction → cleaner backward pass
  3. Empirically: negligible quality difference at 70B+ scale

PRODUCTION IMPLICATION:
  Both are deeply memory-bound (AI < 2). They MUST be fused with adjacent
  GEMM or elementwise ops — standalone RMSNorm wastes 99%+ of compute pipeline.
  Unfused RMSNorm at 80 layers × 10,000 req/s = wasted HBM round-trips that
  compound across the entire serving fleet.
```

2.5 RoPE and Positional Encoding

Method	Used By	Context Extrapolation	FLOPs	Status
Learned absolute	GPT-2, BERT	Hard limit at training length	Zero	Legacy — not used in new models
RoPE	LLaMA, Mistral, Gemma, Qwen	Good with YaRN/LongRoPE (2–4×)	~2% of attention FLOPs	**Production standard [SHIPPED]**
ALiBi	MPT, BLOOM	Excellent linear decay bias	Minimal	Declining — RoPE preferred

RoPE encodes position by rotating Q and K vectors: $Q_m = R_m \times Q$, $K_n = R_n \times K$. The inner product QK^T depends only on relative position $(m-n)$, giving relational information without absolute embeddings. YaRN and LongRoPE extend effective context 2–4× beyond training length by modifying rotation frequency interpolation.

2.6 Feed-Forward Networks and the Activation Bottleneck

```
SWIGLU FFN (LLaMA-3 standard):
  FFN(x) = (SiLU(x·W_gate) ⊗ x·W_up) × W_down
  Three matrices: W_gate, W_up ∈ ℝ^(d_model × d_ffn), W_down ∈ ℝ^(d_ffn × d_model)

  FLOPs per token per layer (LLaMA-3 70B: d_ffn=28,672):
    3 × 2 × 1 × 8192 × 28,672 = 1,409.3M FLOPs

  FFN fraction at decode S=1:
    1409 / (1409 + 302) = 82.4% of total FLOPs

  FFN fraction at prefill S=4096 (attention now contributes):
    1409 / (1409 + 302 + 67) ≈ 79%

  → The FFN is the dominant compute consumer regardless of sequence length.
    INT4 FFN quantization: weight bytes 4× smaller → 4× faster decode (BW-limited).
    FP8 FFN: 2× faster (both BW and compute).
    Quantize FFN first — it gives the highest bandwidth ROI per bit.
```

2.7 Complete FLOP Budget — LLaMA-3 70B

```
FULL FORWARD PASS BUDGET (LLaMA-3 70B, per token):
  Config: L=80, d=8192, H_q=64, H_kv=8, d_head=128, d_ffn=28,672

  Per-layer breakdown:
    Q proj:                134.2M FLOPs
    K proj (GQA H_kv=8):   16.8M FLOPs
    V proj:                 16.8M FLOPs
    Attention scores (S=1):  0.13M FLOPs  (negligible at decode)
    Output proj:           134.2M FLOPs
    FFN SwiGLU:          1,409.3M FLOPs
    ──────────────────────────────────
    Total per layer:      1,711.5M ≈ 1.71 GFLOP
    Total (L=80):         80 × 1.71 = 136.9 GFLOP per token

  BANDWIDTH FLOOR vs COMPUTE FLOOR:
    Compute floor (989 TFLOPS, 100% MFU):  136.9G / 989T = 0.138 ms
    Bandwidth floor (3.35 TB/s, 140 GB BF16 weights): 140G / 3.35T = 41.8 ms

    The bandwidth floor is 303× higher than the compute floor.
    Decode is ALWAYS bandwidth-limited at any batch below the ridge point.
    No kernel optimization can beat the bandwidth floor — only quantization
    or batching changes it.
```

2.8 GQA, MQA, and MLA — Attention Variants

```
KV CACHE REDUCTION VARIANTS (LLaMA-3 70B, S=4096, BF16):

MHA (Multi-Head Attention — baseline):
  H_q = H_kv = 64 heads
  KV bytes = 2 × 80 × 64 × 128 × 4096 × 2 = 42.9 GB per request   ← impractical

GQA (Grouped-Query Attention — LLaMA-3 70B standard):
  H_q = 64, H_kv = 8  (group ratio R = 8)
  KV bytes = 2 × 80 × 8 × 128 × 4096 × 2 = 5.37 GB per request   ← production
  Quality loss: ~0%  (Ainslie et al. 2023 — empirically verified at 70B+ scale)
  Serving impact: 8× more concurrent requests per GPU vs MHA

MQA (Multi-Query Attention — extreme):
  H_q = 64, H_kv = 1
  KV bytes = 2 × 80 × 1 × 128 × 4096 × 2 = 0.67 GB per request
  Quality loss: ~1–2% on complex reasoning tasks

MLA (Multi-head Latent Attention — DeepSeek-V3):
  Compress KV to low-rank latent c_KV ∈ ℝ^(d_c), d_c = 512
  KV bytes ≈ d_c × 2 = 1,024 bytes/token/layer → 32× reduction vs MHA
  Trade-off: K/V reconstruction GEMM at decode (compute cost vs BW savings)
  Break-even: reconstruction cost amortized at batch ≈ 8+
```



Figure 2.2 — KV cache memory comparison at S=4096 context, LLaMA-3 70B, BF16. GQA delivers 8× reduction over MHA with negligible quality loss, enabling 8× more concurrent users per GPU. MLA (DeepSeek) achieves ~33× reduction over MHA but requires a reconstruction GEMM at each decode step.

2.9 Mixture-of-Experts Architecture

```
MOE ARCHITECTURE (Mixtral 8×7B example):
  N = 8 experts, K = 2 (top-K routing per token)
  Each expert FFN: d_ffn = 14,336  (same as a dense 7B baseline)

  Active FLOPs per token: K/N × N × FFN_flops = 2 × FFN_7B ≈ same as dense 7B
  Total parameters: 8 × FFN + shared_layers ≈ 47B total, 14B active

  QUALITY-PER-FLOP ARGUMENT:
    Same active FLOPs as dense 7B but access 8× more parameter knowledge.
    Each token routed to 2 specialist experts → conditional computation.
    Result: quality closer to dense 47B at compute cost of dense 14B.

  SYSTEM COST: All-to-All collective at every FFN layer
    Volume per layer = batch × d_model × 2 bytes (token routing)
    At batch=32: 32 × 8192 × 2 = 524 KB per layer per All-to-All
    Over IB NDR (50 GB/s): 524KB / 50GB/s = 0.01ms → acceptable
    Expert load imbalance: catastrophic if routing collapses (all tokens → 1 expert)
    Auxiliary loss (load balancing loss) is required to prevent this.
```

Chapter 2 Quick Reference

Quantity	Formula	LLaMA-3 70B Value
Q projection FLOPs	2 × d_model × H_q × d_head	134.2M per token per layer
KV projection FLOPs (GQA)	2 × 2 × d_model × H_kv × d_head	33.6M per token per layer
FFN FLOPs (SwiGLU)	3 × 2 × d_model × d_ffn	1,409M per token per layer
FFN fraction (decode S=1)	FFN_flops / total_flops	**82.4%**
Total FLOPs per token	L × (QKV + O + FFN)	136.9 GFLOP (80 layers)
KV cache per token per layer	2 × H_kv × d_head × dtype_bytes	4 KB (BF16, GQA)
KV cache per request (4K ctx)	L × S × 4 KB	5.37 GB (BF16)
TPOT floor (bandwidth)	model_bytes / HBM_BW	41.8ms (H100, BF16)
GQA reduction vs MHA	H_kv_GQA / H_kv_MHA	8× (64 → 8 heads)

Chapter 2 Review Questions

0. **In plain language, explain why a transformer with GQA (H_kv=8) can serve more concurrent users than MHA (H_kv=64), even though both produce the same output quality.**
   Compute the exact maximum concurrent requests on a single H100 (80 GB) for LLaMA-3 70B (TP=2, 35 GB weights per GPU, 5 GB runtime overhead) at 4096 context for MHA vs GQA. How does INT8 KV quantization change this?

0. **Why does FFN dominate FLOPs at 83% during decode but the fraction decreases during long-context prefill?**
   At what sequence length S do attention score FLOPs ($4 \times S^2 \times H_q \times d_{head}$ per layer) exceed FFN FLOPs ($3 \times 2 \times d_{model} \times d_{ffn}$ per layer) for LLaMA-3 70B? Show the calculation and explain the architectural implication.

0. **A team is choosing between GQA (H_kv=8), MQA (H_kv=1), and MLA (d_c=512) for a new 13B model targeting 100K token context length.**
   Compute KV cache bytes per request at S=100K in BF16 for each variant. At what serving scale (concurrent requests) does the per-GPU KV budget become the bottleneck, assuming 40 GB available for KV on H100? Which variant do you recommend and why?

0. **MLA stores a compressed latent c_KV ∈ ℝ^(d_c) instead of K and V. During decode, K and V are reconstructed via two small GEMMs. Is this reconstruction compute-bound or memory-bound on H100 at batch=1?**
   Compute the arithmetic intensity of the reconstruction GEMM (c_KV × W_UK, shape [1, d_c=512] × [512, H_q×d_head=16384]) and compare to the H100 ridge point. At what batch size does it cross the ridge?

0. **A MoE model (8 experts, top-2, d_ffn=14336 per expert, d_model=7168) processes batch=128 at inference.**
   (a) Active FLOPs per token vs dense baseline. (b) All-to-All volume per FFN layer in bytes. (c) At 50 GB/s IB NDR, what is the All-to-All latency? Is MoE routing a bottleneck at this scale? (d) What happens to routing quality when the load balancing loss weight is set to zero?

0. **Design an inference architecture for LLaMA-3 70B serving 5,000 concurrent 100K-token context requests with P99 TPOT < 50ms. Quantify the KV cache memory requirement and the minimum GPU fleet size.**
   Compare: (a) BF16 weights + BF16 KV, H100 TP=2; (b) FP8 weights + INT8 KV, MI300X single GPU. Show all calculations, state all assumptions, and justify the recommended configuration with a 3-year TCO estimate.

Chapter Cross-References

Ch 1 Roofline framework — the arithmetic intensity calculations in §2.2 and §2.7 apply directly to Ch 1's regime classification. GEMV decode at AI≈1 is memory-bound; large GEMM prefill at AI≈1365 is compute-bound.

Ch 4 FlashAttention — §2.2 establishes why standard attention is always memory-bound (AI≈d/3, independent of S). Ch 4 derives the exact HBM traffic formula and shows how FA eliminates the S^2 writes.

Ch 6 Inference architecture — the KV cache math in §2.3 (5.37 GB per request at 4K context) drives the PagedAttention motivation, disaggregated serving design, and fleet sizing formula in Ch 6.

Ch 11 KV cache engineering — the per-request KV sizing from §2.3 and attention variant tradeoffs from §2.8 are the input to Ch 11's paging, eviction, prefix caching, and quantization strategies.

Ch 13 Speculative decoding and MoE — §2.9 introduces MoE architecture. Ch 13 covers the expert routing All-to-All collective, load balancing failure modes, and production MoE serving with disaggregated experts.